Group 130 Assignment 2 Report

Khayaam Waris - kw848, Luca Piergentili - lp2022, Vincent Morel Cavailhes - vc466

Summary

Stratospheric balloons are commonly used to take high-altitude data readings for scientific projects, but due to their underactuation, they are difficult to control. A model of a helium weather balloon's 2D trajectory was created using MATLAB, which considered the wind velocities and atmospheric conditions at varying altitudes. The balloon had an approximate total mass of 29 kg, including 10 kg of sand ballast that could be jettisoned at any moment during the balloon's flight and approximately 4 kg of Helium in the balloon's envelope, which could be vented. The balloon must travel from a vertical distance of 18 km to 15 km whilst covering a horizontal distance of 3 km. Using free body diagrams, Newton's second law and some underlying geometrical and physical assumptions, a set of Ordinary Differential Equations (ODEs) were derived and used to model the balloon's trajectory. To reach the desired target point, 18.64 moles of helium were vented from the balloon's envelope. This solution was evaluated using the RK4 method with a time step of 1 second.

Certain assumptions regarding the balloon's horizontal displacement were omitted to enhance the model. More precise Ordinary Differential Equations (ODEs) were formulated to consider horizontal air resistance while maintaining a consistent amount of vented Helium. A Graphical User Interface (GUI) was also incorporated to streamline user interaction with the model.

1) Introduction

Weather balloons represent a cost-effective and straightforward apparatus capable of conducting high-altitude measurements and tests. Their operational principle relies solely on the buoyancy force of Helium, allowing them to ascend to the upper reaches of the atmosphere. As a result of their simplicity, they are very hard to control, requiring accurate simulations of the weather, buoyancy forces and atmospheric changes due to altitude[1].

A start-up remote-sensing company is creating a weather balloon that will capture data about the earth's surface; the objective is to develop a MATLAB-based model for the trajectory of the balloon, subsequently utilized for guiding the balloon to a predefined target location.

2) Theory

2.1) Given Data and Task

The Balloon's envelope is composed of Latex and has a mass of 10 kg. 1,000 moles of Helium are contained within and provide the upward buoyancy force. The balloon's payload has a mass of 5 kg and 10 kg of sand ballast. Sand and Helium can be released at any point during the balloon's journey. This can be done to alter the balloon's trajectory to reach a desired target. **Figure 1** below illustrates the balloon as well as its starting and final positions (not to scale).

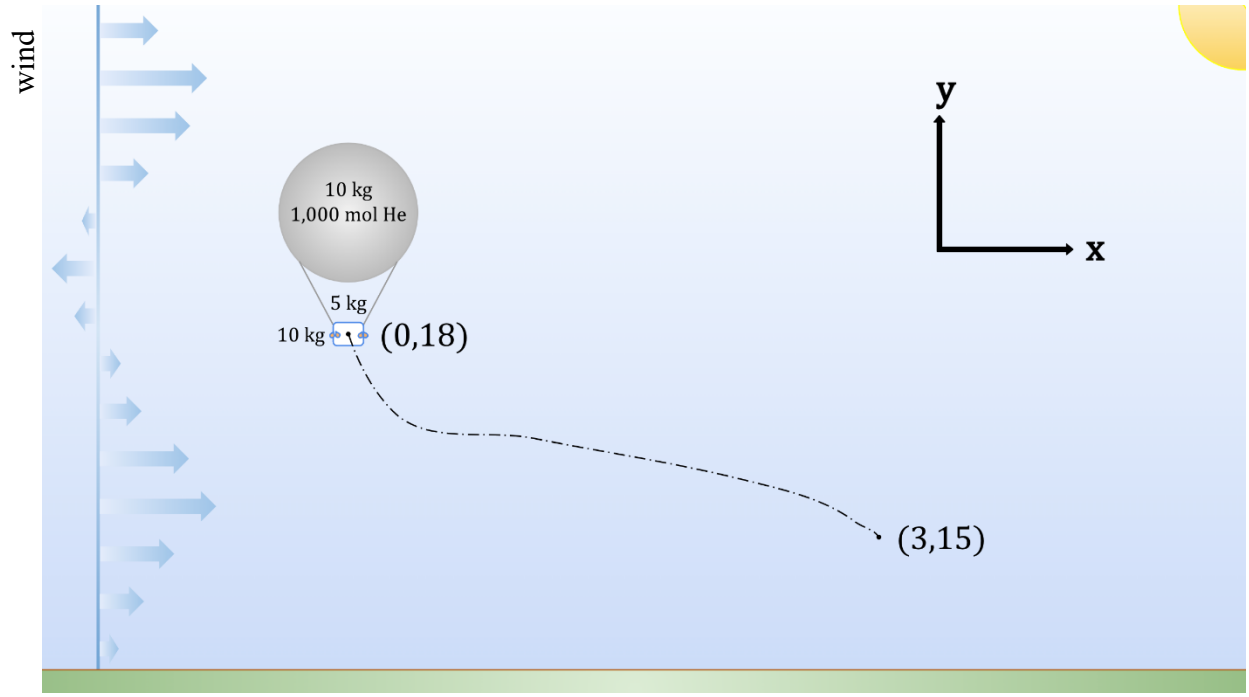


Figure 1: Illustration of the balloon's trajectory.

The balloon must navigate from an altitude of 18 km to an altitude of 15 km while covering a horizontal distance of 3 km. The task is to develop and test a model of the balloon's trajectory to determine the amount of sand that needs to be dropped or the amount of Helium that needs to be vented in order to perform this maneuver.

2.2) Assumptions

The main assumption was that the balloon's envelope was a perfect sphere, leading to the vertical cross-section (used to calculate air resistance) being a perfect circle. The gas inside of the envelope was assumed to be a perfect gas, and only two directions (vertical and horizontal) of motion were considered.

2.3) Derivation of the Ordinary Differential Equations (ODEs)

The forces on the balloon can be split into vertical and horizontal components; the total balloon weight, buoyancy force and air resistance affect the balloon's vertical motion. The forces due to wind velocity, affect its horizontal motion. Figure 2 below shows the free-body diagram of the balloon:

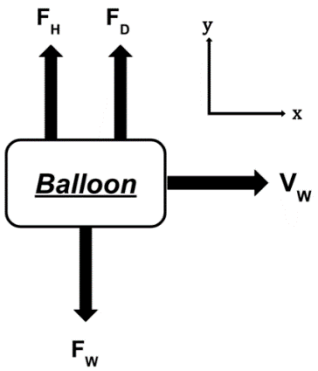


Figure 2: Free-body diagram of the balloon.

The balloon's total weight force can be expressed as:

$$F_W = (M_B + M_P + M_S + M_H) \times g \quad (1)$$

Where:
M_B is the mass of the balloon's envelope.
M_P is the mass of the balloon's payload.
M_S is the mass of the sand ballast.
M_H is the mass of helium in the balloon's envelope.
g is the gravitational acceleration $= 9.81\ ms^{-1}$.

M_T denotes the total mass of the balloon and was used in the code and derivation for the sake of simplicity. The balloon's buoyancy force F_H can be expressed as:

$$F_H = \frac{R \times g \times n \times \rho_{air} \times T}{P} \quad (2)$$

Where:
R is the gas constant = 8.3415 J/molK
g is gravitational acceleration = 9.81 ms^{-2}
n is the number of Helium moles in the balloon's envelope.
ρ_{air} is the outside air density.
T is the outside air temperature.
P is the outside air pressure.

Altitude induces variations in ρ_{air}, T, and P due to atmospheric effects; a function has been provided to compute these values when altitude is specified as an input. This increased the accuracy of the model. The force due to air resistance as the balloon moves through the atmosphere can be expressed as:

$$F_D = \frac{1}{2} \times \rho_{air} \times u^2 \times C_D \times A \quad (3)$$

Where:
u is the balloon's vertical velocity
C_D is the balloon's coefficient of drag $= 0.25$
A is the balloon's vertical cross-sectional area

Air resistance always opposes the direction of motion, hence why it is acting vertically upwards. Using Newton's second law, a set of ODEs were derived as shown below.

$$F_T = F_D + F_H - F_W \quad (4)$$

$$F_T = M_T \times a = \frac{1}{2}\rho_{air} u^2 C_D A + \frac{Rgn\rho_{air}T}{P} - (M_B + M_P + M_S + M_H)g \quad (5)$$

$(M_B + M_P + M_S + M_H)$ was simplified to M_T and acceleration a and velocity u were written as $\ddot{y}$ and $\dot{y}$ respectively. The equation was re-arranged to solve for $\ddot{y}$ and then was put into first order from.

$$M_T\ddot{y} = \frac{1}{2}\rho_{air}(\dot{y})^2 C_D A + \frac{Rgn\rho_{air}T}{P} - M_T g \quad (6)$$

$$\ddot{y} = \frac{\rho_{air} C_D A}{2M_T}(\dot{y})^2 + \frac{Rgn\rho_{air}T}{PM_T} - g \quad (7)$$

$$\begin{cases} z_1 = y \\ z_2 = \dot{y} \end{cases} \tag{8}$$

$$\begin{cases} dz_1 = \dot{y} \\ dz_2 = \ddot{y} \end{cases} \tag{9}$$

$$\begin{cases} dz_1 = \dot{y} \\ dz_2 = \dfrac{\rho_{air} C_D A}{2M_T}(\dot{y})(|\dot{y}|) + \dfrac{RgnP\rho_{air}T}{PM_T} - g \end{cases} \tag{10}$$

Instead of squaring $\dot{y}$, $\dot{y}$ was multiplied by its absolute value to maintain its sign but still output the squared quantity of $\dot{y}$. This was done as the sign of $\dot{y}$ affects whether F_D acts upwards or downwards. These two equations were then inputted into the "StateDeriv.m" function to be able to incrementally calculate the balloon's vertical displacement relative to its starting point for any given time. As far as the balloon's horizontal displacement is concerned, it was assumed that the balloon's horizontal velocity matched the wind's horizontal at the given height. For example, if the wind velocity was 5 ms^{-1} at an altitude of 17 km, the balloon would have a horizontal velocity of 5 ms^{-1} when floating at an altitude of 17 km (positive displacement being towards the right). The wind velocities at any altitude were given by two functions: "importWindForecast1D.m" and "evaluateWindForecast1D.m".

3) Final Balloon Trajectory

As the balloon's buoyancy force was greater than its weight force, the balloon would float upwards until it reached the edge of the atmosphere. To counteract this, a total of 18.64 moles of Helium were vented at the beginning of the maneuver and the maneuver lasted 1184 seconds. There was an error in the final balloon's position as the balloon was off by 0.21 meters in the vertical direction compared to the intended target. No error was made in the horizontal direction as the code stops running after the balloon travels 3000 meters. The final balloon trajectory can be seen in Figure 3 below:

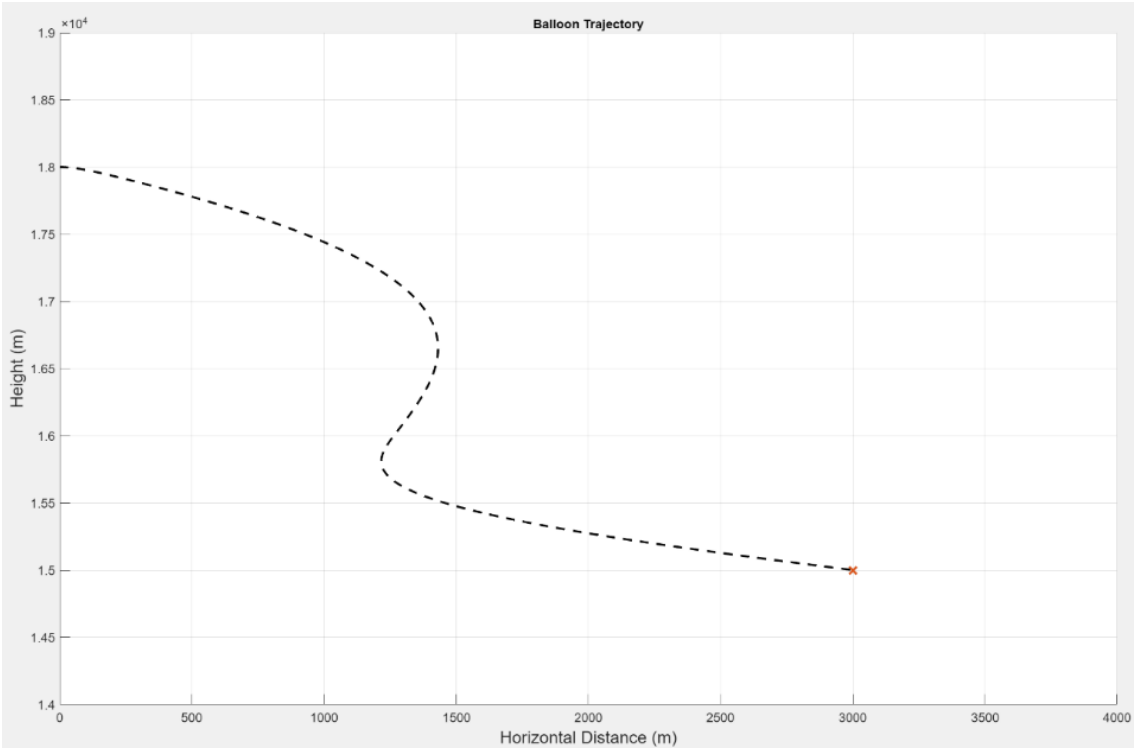


Figure 3: Balloon Trajectory for Initial Task

4) Discussion

4.1) Basic Task Code

The first step was to create a model of the balloon and its behaviour in the atmosphere. The problem was considered a Boundary Value Problem (BVP) and the function “ivpSolver” was adapted to model the balloon. “ivpSolver” creates an array of values that correspond to the balloon’s motion at every time step. To calculate those values, the function “stepRungaKutta” and “stateDeriv” are used. “stateDeriv” contains the ODEs which are used to calculate the balloon’s acceleration and velocity at a given time. Some of the variables in the ODEs change with altitude due to atmospheric effects, functions were used to calculate these values at the balloon’s current iteration altitude. Those values are then fed into “stepRungaKutta”, which uses the RK4 method to compute the next iteration of the distances and velocities of the balloon. This process iterates through itself until the balloons position reaches the target co-ordinates. Once this is achieved the balloons flight path is sent to the GUI along with other display variables, and the trajectory of the balloon is plotted and displayed. Figure 4 shows a flow diagram of the code:

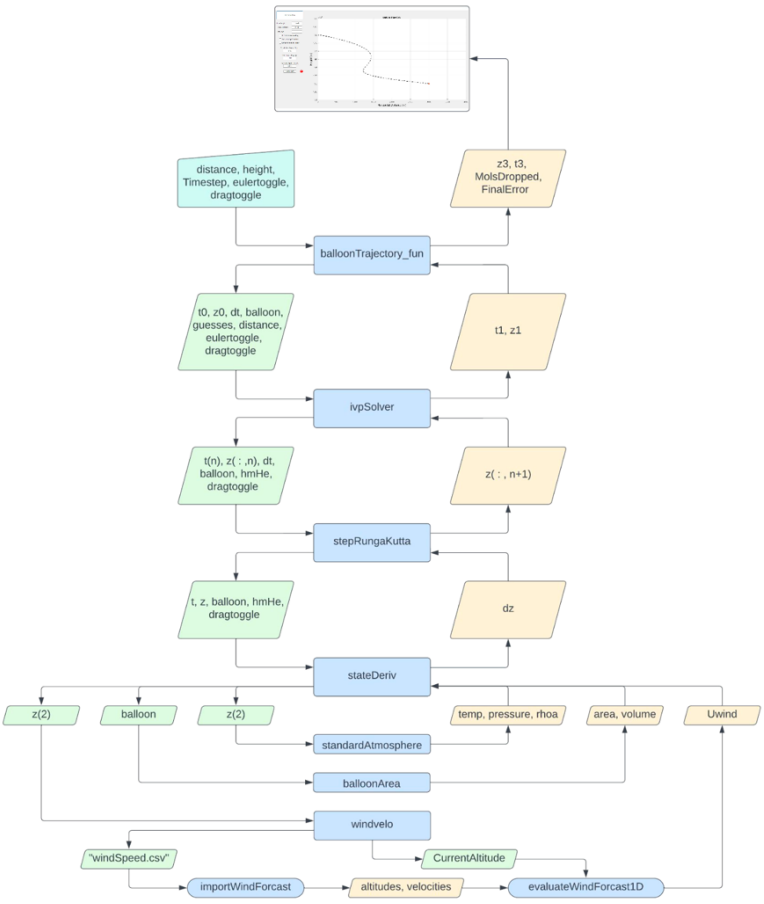


Figure 4: Flow diagram of the code.

To calculate the amount of helium that must be vented to reach the target, the Secant shooting method was implemented. The Secant method works by taking two initial guesses for the amount of helium that has to be vented, it then evaluates the balloons trajectory using these guesses and calculates their final height error, a third guess is then calculated as an average of the two previous guesses and the final error is re-evaluated. This process is done iteratively until the error falls below a set amount which determines the accuracy desired by the user. The way this method was implemented was by setting the guesses as the amount of helium vented, or sand dropped, and then for the code to generate the balloon trajectory with that guess. The error (distance of the balloon from the desired arrival point) is then

calculated and another guess is made which is then used to run the code again. This process repeats until a final guess with an acceptable error is computed. This can be seen in Figure 5 below:

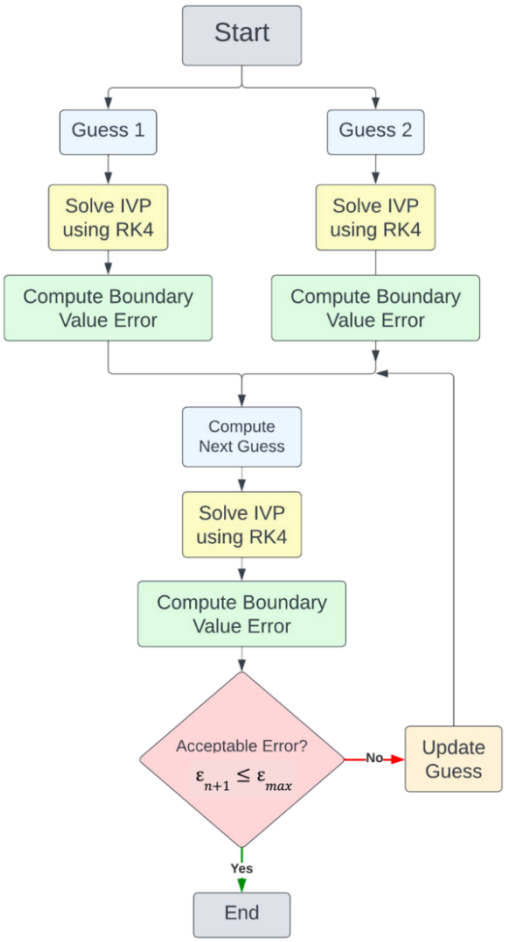


Figure 5: Flow diagram of the Secant Method

4.2) Extension tasks

4.2.1) Accounting for Horizontal Drag

The first extension task tackled was to no longer assume the balloon's velocity matches the wind velocity for a given height, this meant a new balance of forces (wind force and air resistance) had to be computed. Force due to wind velocity can be expressed as shown in equation 11, u_w being the wind velocity[2]:

$$F_w = u_w \times A \times \rho \tag{11}$$

Newton's second law was again used to derive an ODE in the x-direction:

$$\begin{cases} dz_3 = \dot{x} \\ dz_4 = \dfrac{A\rho_{air}[(u_w)^2 - 0.125(\dot{x})(|\dot{x}|)]}{M_T} \end{cases} \tag{12}$$

The original assumption argued that air resistance was non-existent as in the air's frame of reference, the balloon is stationary as both the air and the balloon are being pushed by the wind. But as the balloon descends, the wind speed and direction changes and the forces on the balloon change with time, leading

to different horizontal accelerations and velocities. This slightly impacted the balloon trajectory but not the amount of Helium vented, which remained: 18.64 mols.

4.2.2) Creating a Graphical User Interface

The second extension task involved implementing a General User Interface (GUI) into the code to help the user use the code. The user can input the target height and distance and the GUI will display a graph of the balloon's trajectory and the amount of Helium vented. The GUI has also been ornamented with an animation of the balloon travelling along the modelled path and a progress bar. The GUI also displays the amount of helium vented, mass of sand dropped, vertical error from target and the total time of flight. These outputs both help the user get a better understanding of the trajectory simulation and assisted in testing and debugging the code.

4.2.3) Generalising the program for upward trajectories

The code at this point was unable to accurately calculate the trajectory of the balloon for altitudes greater than the starting position. To fix this a secondary shooting method was implemented that calculated the amount of sand to be dropped instead of helium to be vented. This allowed a more accurate upwards trajectory to be plotted. However, there are limits to this process. At target positions with an altitude greater than 50 km and large horizontal displacements, the code was unable to calculate the balloons path. This was because the programme assumes that any ejection of sand or helium occurs instantaneously, and as the wind velocity distribution contains lower velocities at higher altitudes the balloon is unable to remain at a high speed, this limits the maximum horizontal distance that can be reached for higher altitudes. To solve this issue the program could be edited to allow the sand/helium to be ejected at any time within the balloons flight as well as instantaneously.

4.2.4) Comparing RK4 and Euler's

The third extension task tackled was to compare Euler's method to the RK4 method when computing the Balloon's path. The GUI displays the modelled paths computed using the two different methods. Euler's method for solving ODEs is less accurate than RK4 for a given time step. Solutions using the Euler method become unstable earlier compared to RK4 when using increasing time steps to compute a solution. This is a result of the decreased accuracy using Euler as can be seen by the difference in paths shown in Figure 6.

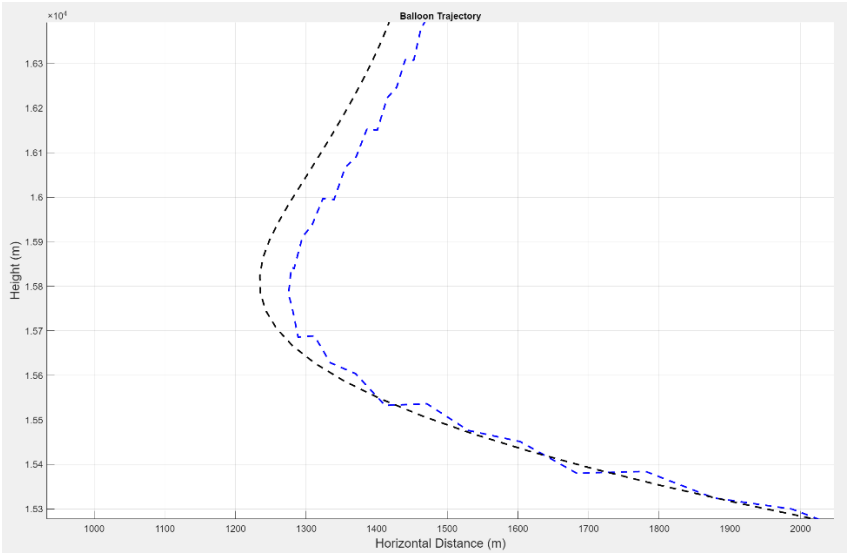


Figure 6: Difference in the RK4 method (black line) and the Euler's method (blue line)

The difference in amount of Helium vented which came out to be: 18.59 mols with a time step of 1. The Euler method was more accurate than RK4, obtaining an error of only + 0.05 meters, compared to RK4's error of - 0.21 meters. This can be explained by utilising the secant method to compute the solution, this method negates the difference in accuracy of the two methods and generates a "random" error. Further evidence for this phenomenon, is that a smaller error was sometimes achieved with larger time steps. At larger time steps, it was observed that the RK4 balloon computed trajectory was smoother than the Euler solution, as expected due to RK4's higher level of accuracy at larger time steps.

5) Potential Improvements

ODEs derived for this assignment do not consider the balloon's momentum, and instead assume that it instantly changes velocity with the wind. To increase the model's accuracy, the balloon's momentum could be accounted for by modifying the ODEs used to model the balloon.
The model assumes that all the helium and sand are ejected instantaneously, another potential improvement would be to allow the program to vent helium and drop sand at any time during the flight. If this feature is implemented, it would allow simulation to reach target points located further away, by making use of higher wind speeds at lower altitudes. Furthermore, should a disturbance in the balloon's trajectory happen during its flight, the code can it into account and correct the balloon's path to reach the target position.
Lastly, the wind velocities are currently given for altitudes less than 50,000 meters which means the balloons trajectory can only be calculated for values below that given altitude. To improve this, a weather forecast giving wind velocities for higher altitudes could be used.

6) Conclusion

This project aimed to develop a MATLAB software which modelled and controlled the trajectory of a stratospheric balloon. The ODEs derived from the FBD as well as the RK4 method and the secant method were used to calculate the amount of helium that must be vented to reach the target position (3, 15). As a result, 18.64 mols of helium had to be vented to reach the basic tasks target position in a timeframe of 1184s with a timestep of 1s. The effect of drag on the horizontal was also examined using separately derived ODEs, however no visible effect on the amount of helium vented or the path of the balloon was observed, thus indicating the effect of drag was negligible. A GUI was also created which enabled the user to easily edit the desired target position, view the error from the target height as well as the option to utilise the Euler method instead of the RK4 method. This feature was used to compare the accuracy between the two methods; though the RK4 method had a smoother trajectory at higher timesteps than the Euler method it was found that the RK4 method was not always more accurate than the Euler method as theory suggested. This effect was due to the use of the secant method which introduces its own randomness into the solution.

7) References

1) M.G.Bellemareetal.,"Autonomousnavigationofstratosphericballoonsusingreinforcement learning",Nature 588,77–82,2020. https://doi.org/10.1038/s41586-020-2939-8.
2) G Blaettler, K. (2020). *How to Convert Wind Speed to Force*. [online] Sciencing. Available at: https://sciencing.com/convert-wind-speed-force-5985528.html.

8) Appendix: MatLab Code

Graphical User Interface

```
classdef BalloonTrajectoryUserInterface < matlab.apps.AppBase

    % Properties that correspond to app components
    properties (Access = public)
        UIFigure                        matlab.ui.Figure
        GridLayout                      matlab.ui.container.GridLayout
        LeftPanel                       matlab.ui.container.Panel
        TotalsanddroppedKgEditField     matlab.ui.control.NumericEditField
        TotalsanddroppedKgEditFieldLabel  matlab.ui.control.Label
        ConsiderhorizontaldragCheckBox  matlab.ui.control.CheckBox
        VerticalHeightErrormEditField   matlab.ui.control.NumericEditField
        VerticalHeightErrormEditFieldLabel  matlab.ui.control.Label
        TotalTimeofFlightsEditField     matlab.ui.control.NumericEditField
        TotalTimeofFlightsEditFieldLabel  matlab.ui.control.Label
        RunningLamp                     matlab.ui.control.Lamp
        AnimatemotionofBalloonCheckBox  matlab.ui.control.CheckBox
        ClearFigureButton               matlab.ui.control.Button
        TotalMolsofHeliumDropEditField  matlab.ui.control.NumericEditField
        TotalMolsofHeliumDropLabel      matlab.ui.control.Label
        TimeStepEditField               matlab.ui.control.NumericEditField
        TimeStepEditFieldLabel          matlab.ui.control.Label
        PlotEulerapproximationCheckBox  matlab.ui.control.CheckBox
        FinalxDistanceEditField         matlab.ui.control.NumericEditField
        FinalxDistanceEditFieldLabel    matlab.ui.control.Label
        FinalHeightEditField            matlab.ui.control.NumericEditField
        FinalHeightEditFieldLabel       matlab.ui.control.Label
        RunSimulationButton             matlab.ui.control.Button
        RightPanel                      matlab.ui.container.Panel
        UIAxes                          matlab.ui.control.UIAxes
    end

    % Properties that correspond to apps with auto-reflow
    properties (Access = private)
        onePanelWidth = 576;
    end

    % Callbacks that handle component events
    methods (Access = private)

        % Button pushed function: RunSimulationButton
        function RunSimulationButtonPushed(app, event)
            %changing text of run simulation box to processing once clicked
            app.RunSimulationButton.Text = 'Processing...';

            %changing position of text in button
            app.RunSimulationButton.IconAlignment = 'bottom';
            % changing the lamp colour to green to indicate the
            app.RunningLamp.Color = [0 1 0];

            %creation of the black loading bar outline to appear within
            %the button
            wbar =
permute(repmat(app.RunSimulationButton.BackgroundColor,15,1,200),[1,3,2]);
```

```
            wbar([1,end],:,:) = 0;
            wbar(:,[1,end],:) = 0;
            app.RunSimulationButton.Icon = wbar;

            %taking inputs from user for final height, distance and time
            %step
            drawnow;
            Height = app.FinalHeightEditField.Value;
            distance = app.FinalxDistanceEditField.Value;
            TimeStep = app.TimeStepEditField.Value;

            %checking to see if the user wants to plot the euler
            %approximation
            euler = app.PlotEulerapproximationCheckBox.Value;
            eulertoggle = 0;
            plotColor = "black";

            % checking to see if the user has ticked the euler
            % approximation checkbox
            if euler
                eulertoggle = 1;
                plotColor = "blue";
            end

            %checking to see if the user has ticked the horizontal drag
            %checkbox
            dragmodel = app.ConsiderhorizontaldragCheckBox.Value;
            dragtoggle = 0;
            if dragmodel
                dragtoggle = 1;
            end




            %running through BVP solver using the user inputted values as
            %the final height and x distance
            [z3,MolsDropped,t3,FinalError,SandDropped] =
balloonTrajectory_fun(distance,Height,TimeStep,eulertoggle,dragtoggle);


            %plotting graph and adjusting axis/plot properties.
            hold(app.UIAxes,"on")

plot(app.UIAxes,z3(1,:),z3(2,:),"Color",plotColor,"LineWidth",2,"LineStyle","--");

plot(app.UIAxes,distance,Height,"Marker","x","MarkerFaceColor","red",MarkerSize=10,LineW
idth=2);

            app.UIAxes.FontSize = 16;
            title(app.UIAxes,"Balloon Trajectory",FontSize=16);
            ylabel(app.UIAxes,"Height (m)",FontSize=16);
            xlabel(app.UIAxes,"Horizontal Distance (m)",FontSize=16);

            %displaying the amount of helium dropped in the UI
            app.TotalMolsofHeliumDropEditField.Value = MolsDropped;

            app.TotalsanddroppedKgEditField.Value = SandDropped;
            %displaying the time taken for the journey in the UI
```

```
            app.TotalTimeofFlightsEditField.Value = t3(end);
            animated = app.AnimatemotionofBalloonCheckBox.Value;
            app.VerticalHeightErrormEditField.Value = FinalError;

            %initialising the balloon image and reading the jpg file for animation
            balloonHandle = flip(imread("balloonimg.jpg"));

            %if user ticked the animation box then the balloon is produced
            %as an image along the points generated, in turn acting as an
            %animation

            if animated
                for q = 1:length(z3(1,:))

                    %increasing progress of loading bar in fractions as the
                    %loop iterates through the length of the z vector
                    currentProg = min(round((size(wbar,2)-
2)*(q/length(z3(1,:)))),size(wbar,2)-2);
                    RGB = app.RunSimulationButton.Icon;
                    RGB(2:end-1, 2:currentProg+1, 1) = 0.25391;
                    RGB(2:end-1, 2:currentProg+1, 2) = 0.41016;
                    RGB(2:end-1, 2:currentProg+1, 3) = 0.87891;
                    app.RunSimulationButton.Icon = RGB;

                    %the image of the ballon is plotted at the current
                    %(x,y) co-ordinate given by the z vector
                    ba = image(app.UIAxes,z3(1,q)-200,z3(2,q)-200,balloonHandle);
                    drawnow %update the UI
                    pause(0.005);

                    % delete the previous image as to not have stacked images on the
graph
                    delete(ba);

                end
                % if the animation box is not ticked then the progress bar
                % is still shown in order to indicate to the user that the
                % code is running in the background
            else
                for i = 1:length(z3(1,:))
                    currentProg = min(round((size(wbar,2)-
2)*(i/length(z3(1,:)))),size(wbar,2)-2);
                    RGB = app.RunSimulationButton.Icon;
                    RGB(2:end-1, 2:currentProg+1, 1) = 0.25391;
                    RGB(2:end-1, 2:currentProg+1, 2) = 0.41016;
                    RGB(2:end-1, 2:currentProg+1, 3) = 0.87891;

                    app.RunSimulationButton.Icon = RGB;
                end
            end

            %changing the runnning simulation button back to its original
            %state
            app.RunSimulationButton.Icon = '';
            app.RunSimulationButton.Text = "Run simulation";
            hold(app.UIAxes,"off")

            %revering the lamp colour to red
            app.RunningLamp.Color = [1 0 0];
```

```
        end

        % Button pushed function: ClearFigureButton
        function ClearFigureButtonPushed(app, event)
            cla(app.UIAxes);
        end

        % Changes arrangement of the app based on UIFigure width
        function updateAppLayout(app, event)
            currentFigureWidth = app.UIFigure.Position(3);
            if(currentFigureWidth <= app.onePanelWidth)
                % Change to a 2x1 grid
                app.GridLayout.RowHeight = {513, 513};
                app.GridLayout.ColumnWidth = {'1x'};
                app.RightPanel.Layout.Row = 2;
                app.RightPanel.Layout.Column = 1;
            else
                % Change to a 1x2 grid
                app.GridLayout.RowHeight = {'1x'};
                app.GridLayout.ColumnWidth = {239, '1x'};
                app.RightPanel.Layout.Row = 1;
                app.RightPanel.Layout.Column = 2;
            end
        end
    end

    % Component initialization
    methods (Access = private)

        % Create UIFigure and components
        function createComponents(app)

            % Create UIFigure and hide until all components are created
            app.UIFigure = uifigure('Visible', 'off');
            app.UIFigure.AutoResizeChildren = 'off';
            app.UIFigure.Position = [100 100 668 513];
            app.UIFigure.Name = 'MATLAB App';
            app.UIFigure.SizeChangedFcn = createCallbackFcn(app, @updateAppLayout,
true);

            % Create GridLayout
            app.GridLayout = uigridlayout(app.UIFigure);
            app.GridLayout.ColumnWidth = {239, '1x'};
            app.GridLayout.RowHeight = {'1x'};
            app.GridLayout.ColumnSpacing = 0;
            app.GridLayout.RowSpacing = 0;
            app.GridLayout.Padding = [0 0 0 0];
            app.GridLayout.Scrollable = 'on';

            % Create LeftPanel
            app.LeftPanel = uipanel(app.GridLayout);
            app.LeftPanel.BackgroundColor = [0.9412 0.9412 0.9412];
            app.LeftPanel.Layout.Row = 1;
            app.LeftPanel.Layout.Column = 1;

            % Create RunSimulationButton
            app.RunSimulationButton = uibutton(app.LeftPanel, 'push');
            app.RunSimulationButton.ButtonPushedFcn = createCallbackFcn(app,
@RunSimulationButtonPushed, true);
```

```
app.RunSimulationButton.BackgroundColor = [1 1 1];
app.RunSimulationButton.Position = [16 429 208 67];
app.RunSimulationButton.Text = 'Run Simulation';

% Create FinalHeightEditFieldLabel
app.FinalHeightEditFieldLabel = uilabel(app.LeftPanel);
app.FinalHeightEditFieldLabel.HorizontalAlignment = 'right';
app.FinalHeightEditFieldLabel.Position = [14 400 69 22];
app.FinalHeightEditFieldLabel.Text = 'Final Height';

% Create FinalHeightEditField
app.FinalHeightEditField = uieditfield(app.LeftPanel, 'numeric');
app.FinalHeightEditField.HorizontalAlignment = 'center';
app.FinalHeightEditField.Position = [138 400 82 22];
app.FinalHeightEditField.Value = 15000;

% Create FinalxDistanceEditFieldLabel
app.FinalxDistanceEditFieldLabel = uilabel(app.LeftPanel);
app.FinalxDistanceEditFieldLabel.HorizontalAlignment = 'right';
app.FinalxDistanceEditFieldLabel.Position = [15 367 91 22];
app.FinalxDistanceEditFieldLabel.Text = 'Final x Distance';

% Create FinalxDistanceEditField
app.FinalxDistanceEditField = uieditfield(app.LeftPanel, 'numeric');
app.FinalxDistanceEditField.HorizontalAlignment = 'center';
app.FinalxDistanceEditField.Position = [139 367 82 22];
app.FinalxDistanceEditField.Value = 3000;

% Create PlotEulerapproximationCheckBox
app.PlotEulerapproximationCheckBox = uicheckbox(app.LeftPanel);
app.PlotEulerapproximationCheckBox.Text = 'Plot Euler approximation';
app.PlotEulerapproximationCheckBox.Position = [38 275 157 22];

% Create TimeStepEditFieldLabel
app.TimeStepEditFieldLabel = uilabel(app.LeftPanel);
app.TimeStepEditFieldLabel.HorizontalAlignment = 'right';
app.TimeStepEditFieldLabel.Position = [15 333 59 22];
app.TimeStepEditFieldLabel.Text = 'Time Step';

% Create TimeStepEditField
app.TimeStepEditField = uieditfield(app.LeftPanel, 'numeric');
app.TimeStepEditField.HorizontalAlignment = 'center';
app.TimeStepEditField.Position = [139 333 82 22];
app.TimeStepEditField.Value = 3;

% Create TotalMolsofHeliumDropLabel
app.TotalMolsofHeliumDropLabel = uilabel(app.LeftPanel);
app.TotalMolsofHeliumDropLabel.HorizontalAlignment = 'right';
app.TotalMolsofHeliumDropLabel.Position = [47 210 146 22];
app.TotalMolsofHeliumDropLabel.Text = 'Total Mols of Helium Drop ';

% Create TotalMolsofHeliumDropEditField
app.TotalMolsofHeliumDropEditField = uieditfield(app.LeftPanel, 'numeric');
app.TotalMolsofHeliumDropEditField.HorizontalAlignment = 'center';
app.TotalMolsofHeliumDropEditField.Position = [70 188 100 22];

% Create ClearFigureButton
app.ClearFigureButton = uibutton(app.LeftPanel, 'push');
```

```
            app.ClearFigureButton.ButtonPushedFcn = createCallbackFcn(app,
@ClearFigureButtonPushed, true);
            app.ClearFigureButton.Position = [70 6 100 23];
            app.ClearFigureButton.Text = 'Clear Figure';

            % Create AnimatemotionofBalloonCheckBox
            app.AnimatemotionofBalloonCheckBox = uicheckbox(app.LeftPanel);
            app.AnimatemotionofBalloonCheckBox.Text = 'Animate motion of Balloon';
            app.AnimatemotionofBalloonCheckBox.Position = [38 246 162 22];

            % Create RunningLamp
            app.RunningLamp = uilamp(app.LeftPanel);
            app.RunningLamp.Position = [200 6 23 23];
            app.RunningLamp.Color = [1 0 0];

            % Create TotalTimeofFlightsEditFieldLabel
            app.TotalTimeofFlightsEditFieldLabel = uilabel(app.LeftPanel);
            app.TotalTimeofFlightsEditFieldLabel.HorizontalAlignment = 'right';
            app.TotalTimeofFlightsEditFieldLabel.Position = [58 71 123 22];
            app.TotalTimeofFlightsEditFieldLabel.Text = 'Total Time of Flight (s)';

            % Create TotalTimeofFlightsEditField
            app.TotalTimeofFlightsEditField = uieditfield(app.LeftPanel, 'numeric');
            app.TotalTimeofFlightsEditField.HorizontalAlignment = 'center';
            app.TotalTimeofFlightsEditField.Position = [70 50 100 22];

            % Create VerticalHeightErrormEditFieldLabel
            app.VerticalHeightErrormEditFieldLabel = uilabel(app.LeftPanel);
            app.VerticalHeightErrormEditFieldLabel.HorizontalAlignment = 'right';
            app.VerticalHeightErrormEditFieldLabel.Position = [52 111 134 22];
            app.VerticalHeightErrormEditFieldLabel.Text = 'Vertical Height Error (m)';

            % Create VerticalHeightErrormEditField
            app.VerticalHeightErrormEditField = uieditfield(app.LeftPanel, 'numeric');
            app.VerticalHeightErrormEditField.HorizontalAlignment = 'center';
            app.VerticalHeightErrormEditField.Position = [70 90 100 22];

            % Create ConsiderhorizontaldragCheckBox
            app.ConsiderhorizontaldragCheckBox = uicheckbox(app.LeftPanel);
            app.ConsiderhorizontaldragCheckBox.Text = 'Consider horizontal drag';
            app.ConsiderhorizontaldragCheckBox.Position = [38 304 157 22];

            % Create TotalsanddroppedKgEditFieldLabel
            app.TotalsanddroppedKgEditFieldLabel = uilabel(app.LeftPanel);
            app.TotalsanddroppedKgEditFieldLabel.HorizontalAlignment = 'right';
            app.TotalsanddroppedKgEditFieldLabel.Position = [52 164 136 22];
            app.TotalsanddroppedKgEditFieldLabel.Text = 'Total sand dropped  (Kg)';

            % Create TotalsanddroppedKgEditField
            app.TotalsanddroppedKgEditField = uieditfield(app.LeftPanel, 'numeric');
            app.TotalsanddroppedKgEditField.HorizontalAlignment = 'center';
            app.TotalsanddroppedKgEditField.Position = [72 143 100 22];

            % Create RightPanel
            app.RightPanel = uipanel(app.GridLayout);
            app.RightPanel.BorderType = 'none';
            app.RightPanel.Layout.Row = 1;
            app.RightPanel.Layout.Column = 2;
```

```
            % Create UIAxes
            app.UIAxes = uiaxes(app.RightPanel);
            title(app.UIAxes, 'Balloon Trajectory')
            xlabel(app.UIAxes, 'Distance (m)')
            ylabel(app.UIAxes, 'Height (m)')
            zlabel(app.UIAxes, 'Z')
            app.UIAxes.XGrid = 'on';
            app.UIAxes.YGrid = 'on';
            app.UIAxes.Position = [28 24 378 441];

            % Show the figure after all components are created
            app.UIFigure.Visible = 'on';
        end
    end

    % App creation and deletion
    methods (Access = public)

        % Construct app
        function app = BalloonTrajectoryUserInterface

            % Create UIFigure and components
            createComponents(app)

            % Register the app with App Designer
            registerApp(app, app.UIFigure)

            if nargout == 0
                clear app
            end
        end

        % Code that executes before app deletion
        function delete(app)

            % Delete UIFigure when app is deleted
            delete(app.UIFigure)
        end
    end
end
```

Balloon Trajectory Fun

```
function [z3,MolsDropped,t3,FinalError, SandDropped] =
balloonTrajectory_fun(distance,Height,TimeStep,eulertoggle,dragtoggle)
% Calculate balloons trajectory whilst varying the moles of helium and the
% mass of the sand


%% Constants
balloon.R = 8.3145; % Gas Constant (J/molK)
balloon.g = 9.81; % Gravity (ms-1)
balloon.Mb = 10; % Ballon Mass (Kg)
balloon.Mp = 5; % Payload Mass (Kg)
balloon.molarMass = 0.0040026; % Molar mass of Helium (kg/mol)
balloon.dC = 0.25; % Drag Coefficient (Unitless)
balloon.finalDestination = [distance,Height];
```

```
%% Variables
balloon.n= 1000; % Number of moles of helium in the Balloon
balloon.Ms = 10; % Mass of sand (Kg)

%% Initial conditions to run ivpSolver
t0 = 0; % Starting time (s)
dt = TimeStep; % Time step size increase (s)
z0 = [0,18000,0,0]; % Ballon initial conditions [Initial X displacement, Initial Y
displacement, Initial X velocity, Initial Y velocity]

%% Shooting Method
% Creating empty arrays to store the guesses and errors calculated lower
guesses = [];
error = [];
guessesSand = [];
errorSand = [];
n = 1;
MolsDropped = 0;
FinalError = 0;
SandDropped = 0;

if Height < 18000
    % Setting the First Initial Guess for the number of moles of helium needed
    % to be dropped and calculating the error between the actual and required
    % height.
    guesses(1) = 10;
    [t1,z1] = ivpSolver(t0,z0,dt,balloon,guesses(1),distance,eulertoggle,dragtoggle,0);
% Recalling ivpSolver
    error(1) = balloon.finalDestination(2) - z1(2,end);


    % Setting the Second Initial Guess for the number of moles of helium needed
    % to be dropped and calculating the error between the actual and required
    % height.
    guesses(2) = 30;
    [t2,z2] = ivpSolver(t0,z0,dt,balloon,guesses(2),distance,eulertoggle,dragtoggle,0);
% Recalling ivpSolver
    error(2) = balloon.finalDestination(2) - z2(2,end);

    % This while loop takes the two errors calculated above and calculates a
    % new guess for the value of helium needed to be dropped before rerunning
    % ivpSolver to calculate the new error.

    while abs(error) > 1
        n = n+1;
        % Calculates the midpoint guess to get the new value of Helium needed
        % to be dropped and recalculating a new guess
        guesses(n+1) = guesses(n) - error(n)*((guesses(n)-guesses(n-1))/(error(n)-
error(n-1)));
        [t3,z3] =
ivpSolver(t0,z0,dt,balloon,guesses(n+1),distance,eulertoggle,dragtoggle,0);
        error(n+1) = balloon.finalDestination(2) - z3(2,end);
    end
    MolsDropped = guesses(end);
    FinalError = error(end);

elseif Height > 18000
    % Setting the First Initial Guess for the number KG of sand needed
```

```
    % to be dropped and calculating the error between the actual and required
    % height.
    guessesSand(1) = 0;
    [t1,z1] =
ivpSolver(t0,z0,dt,balloon,0,distance,eulertoggle,dragtoggle,guessesSand(1)); %
Recalling ivpSolver
    errorSand(1) = balloon.finalDestination(2) - z1(2,end);


    % Setting the Second Initial Guess for the number of KG of sand needed
    % to be dropped and calculating the error between the actual and required
    % height.
    guessesSand(2) = 9.5;
    [t2,z2] =
ivpSolver(t0,z0,dt,balloon,0,distance,eulertoggle,dragtoggle,guessesSand(2)); %
Recalling ivpSolver
    errorSand(2) = balloon.finalDestination(2) - z2(2,end);

    % This while loop takes the two errors calculated above and calculates a
    % new guess for the value of sand needed to be dropped before rerunning
    % ivpSolver to calculate the new error.

    while abs(errorSand) > 1
        n = n+1;
        % Calculates the midpoint guess to get the new value of Helium needed
        % to be dropped and recalculating a new guess
        guessesSand(n+1) = guessesSand(n) - errorSand(n)*((guessesSand(n)-guessesSand(n-
1))/(errorSand(n)-errorSand(n-1)));
        [t3,z3] =
ivpSolver(t0,z0,dt,balloon,0,distance,eulertoggle,dragtoggle,guessesSand(n+1));
        errorSand(n+1) = balloon.finalDestination(2) - z3(2,end);

        FinalError = errorSand(end);
        SandDropped = guessesSand(end);
    end

end
```

IVP Solver

```
function [t,z] =
ivpSolver(t0,z0,dt,balloon,hmHe,distance,eulertoggle,dragtoggle,droppedSand)
% ivpSolver    Solve an initial value problem (IVP) and plot the result
%
%     [T,Z] = ivpSolver(T0,Z0,DT) computes the IVP solution using a step
%     size DT, beginning at time T0 and initial state Z0. The solution is output as a
time vector T and a matrix of state
%     vectors Z.

% Set initial conditions
t(1) = t0;
z(:,1) = z0;
n=1;

% Continue stepping until the end value is exceeded, if statement decides
% if RK4 or Euler's is used
```

```
if eulertoggle == 0
    while  z(1,n) < distance
        % Increment the time vector by one time step
        t(n+1) = t(n) + dt;

        % Apply RK4 method for one time step
        z(:,n+1) = stepRungaKutta(t(n), z(:,n), dt,balloon,hmHe,dragtoggle,droppedSand);
        n = n+1;
    end
else
    while  z(1,n) < distance
    % Increment the time vector by one time step
        t(n+1) = t(n) + dt;

        % Apply Euler's method for one time step
        z(:,n+1) = stepEuler(t(n), z(:,n), dt,balloon,hmHe,dragtoggle,droppedSand);
        n = n+1;
    end
end
```

Step Runge Kutta

```
function znext = stepRungaKutta(t,z,dt,balloon,hmHe,dragtoggle,droppedSand)
% Compute one step using the Runga Kutta method
%
%  ZNEXT = stepRungaKutta(T,Z,DT) computes the state vector ZNEXT at the next
%  time step T+DT

% Calculate the state derivative from the current state
dz = stateDeriv(t, z,balloon,hmHe,dragtoggle,droppedSand);

% One step of the Runga Kutta Method
A = dt * stateDeriv(t,z,balloon,hmHe,dragtoggle,droppedSand);
B = dt * stateDeriv(t+dt/2,z+A/2,balloon,hmHe,dragtoggle,droppedSand);
C = dt * stateDeriv( t+dt/2,z+B/2,balloon,hmHe,dragtoggle,droppedSand);
D = dt * stateDeriv( t+dt/2,z+C,balloon,hmHe,dragtoggle,droppedSand);
znext = z + (1/6)*(A+2*B+2*C+D);
```

Step Euler

```
function znext = stepEuler(t,z,dt,balloon,hmHe,dragtoggle,droppedSand)
%   Compute one step using the Euler method
%   ZNEXT = stepEuler(T,Z,DT) computes the state vector ZNEXT at the next
%   time step T+DT

% Calculate the state derivative from the current state
dz = stateDeriv(t, z,balloon,hmHe,dragtoggle,droppedSand);

% Calculate the next state vector from the previous one using Euler's
% update equation
znext = z + dt * dz;
```

State Deriv

```
function dz = stateDeriv(t,z,balloon,hmHe,dragtoggle,droppedSand)
```

```
%  Calculate the state derivative for the balloon using coupled ODE's
%  DZ = stateDeriv(T,Z,ballon,hmHe) computes the derivative DZ = [V; A] of the
%  state vector Z = [X; V], where X is displacement, V is velocity,
%  and A is acceleration.

%% Reseting Balloon Constants
balloon.R = 8.3145; % Gas Constant (J/molK)
balloon.g = 9.81; % Gravity (ms-1)
balloon.Mb = 10; % Ballon Mass (Kg)
balloon.Mp = 5; % Payload Mass (Kg)
balloon.molarMass = 0.0040026; % Molar mass of Helium (kg/mol)
balloon.dC = 0.25; % Drag Coefficient (Unitless)

%% Reseting Balloon Variables
balloon.n= 1000 - hmHe; % Number of moles of helium in the Balloon
balloon.Ms = 10 - droppedSand; % Mass of sand (Kg)
balloon.Mh = balloon.n * balloon.molarMass; % Weight of Helium (Kg)
balloon.Mtotal = balloon.Mb +balloon.Mp + balloon.Ms + balloon.Mh ; % Total mass (Kg)

%% Exterior Variables
% Import rhoa, temp, pressure from stamdartAtmosphere
[temp, pressure, rhoa] = standardAtmosphere(z(2));
balloon.temp = temp;
balloon.pressure = pressure;
balloon.rhoa = rhoa;

% Function to get the balloon cross-sectional area and volume at varying
% altitudes
[area,volume] = balloonArea(balloon);

Uwind = windVelo(z(2)); % Velocity of wind at current altitude

Fh = ((rhoa*balloon.g*volume)); % Force due to helium buoyancy
K = (rhoa*balloon.dC*area)/(2*balloon.Mtotal); % Drag constant derived from the ODE's

%% First order Coupled Ode's

d1x = Uwind; % Velocity of the balloon in the X-direction
if dragtoggle == 1
    d2x = (area*rhoa*((Uwind-z(3))^2-0.125*z(3)^2))/balloon.Mtotal;% ODE X-Direction
taking into account drag
else
    d2x = 0; % Acceleration of the balloon in the X-direction
end


d1y = z(4); % Velocity of the balloon in the Y-direction
d2y = -K*(z(4)*abs(z(4))) + (Fh/balloon.Mtotal) - balloon.g; % Acceleration of the
balloon in the Y-direction

dz = [d1x; d1y; d2x; d2y];
```

Balloon Area

```
function [area,volume] = balloonArea(balloon)

%   This function calculates the balloon volume and cross-sectional area at
%   different altitudes using the number of moles in the balloon, the
%   temperature and pressure of the air around the balloon.
```

```
r = ((balloon.n*balloon.R*balloon.temp*3)/(balloon.pressure*4*pi))^(1/3);
%   Find balloon radius from PV=nRT where the volume V was rearranged to
%   make the formula in terms of area.

area = pi*r^2; % Calculates the balloon cross-sectional area

volume = ((4*pi)/3)*r^3; % Calculates the balloon volume.

end
```